

Mini Daily Call Module

API & Database Documentation

Project	Mini Daily Call Module
Backend	Python + Flask + SQLite
Frontend	Flutter (Windows Desktop + Android)
Base URL	http://localhost:5000
Auth Method	JWT Bearer Token

Table of Contents

- PART 1 — API DOCUMENTATION3**
 - 1. Environment Setup3*
 - 2. Demo Accounts3*
 - 3. API Endpoints3*
 - 4. Recommended Testing Flow13*
- PART 2 — DATABASE DOCUMENTATION14**
 - 1. Table Schemas14*
 - Table: users.....14*
 - Table: jobs14*
 - 2. Job Status Flow15*
 - 3. SQL Schema Definitions.....16*
 - 4. How to View the Database16*

PART 1 — API DOCUMENTATION

1. Environment Setup

In Postman, set up your environment by creating two variables:

- **base_url:** `http://localhost:5000`
- **token:** `(leave empty)`

Upon successful login, the token will automatically populate via the Tests tab script.

2. Demo Accounts

User	Email	Password	Role
CS User	cs@demo.com	password123	cs
Manager	manager@demo.com	password123	manager
Tech Ali	ali@demo.com	password123	technician
Tech Reza	reza@demo.com	password123	technician

3. API Endpoints

3.1 Login

Method: POST | **URL:** `{{base_url}}/auth/login` | **Role:** Public (No token required)

Authenticates a user and returns a JWT token. Paste this snippet in the Postman Tests tab to auto-save the token:

```
const res = pm.response.json();
if (res.token) {
  pm.environment.set("token", res.token);
}
```

Headers:

```
Content-Type: application/json
```

Request Body:

```
{
  "email": "cs@demo.com",
  "password": "password123"
}
```

Success Response:

```
{
  "token": "eyJhbGciOi...",
  "user": {
    "id": 1,
    "name": "CS User",
    "email": "cs@demo.com",
    "role": "cs"
  }
}
```

Error Responses:

```
400 Bad Request: {"error": "Email and password required"}
401 Unauthorized: {"error": "Invalid email or password"}
```

3.2 Create Job

Method: POST | **URL:** `{{base_url}}/jobs` | **Role:** CS only

Creates a new field service job. Status defaults to 'pending'. Title and description are required. Location and GPS coordinates are optional.

Headers:

```
Content-Type: application/json
Authorization: Bearer {{token}}
```

Request Body:

```
{
  "title": "Fix generator at Site A",
  "description": "Generator not starting. Check fuel and spark plug.",
  "location": "Jalan Ampang, Kuala Lumpur",
  "latitude": 3.15696,
  "longitude": 101.71149
}
```

Success Response:

```
201 Created
Returns the full job object (id, title, status: 'pending', created_by object,
assigned_to: null, created_at timestamp).
```

Error Responses:

```
400 Bad Request: {"error": "Title and description required"}
403 Forbidden: {"error": "Only CS can create jobs"}
422 Unprocessable Entity: {"error": "Invalid or missing token"}
```

3.3 List Jobs

Method: GET | **URL:** `{{base_url}}/jobs` | **Role:** All roles

Returns jobs filtered by user role. CS and Manager receive all jobs. Technicians only receive jobs assigned to them.

Headers:

```
Authorization: Bearer {{token}}
```

Success Response:

```
200 OK  
Returns an array of job objects.
```

Error Responses:

```
422 Unprocessable Entity: {"error": "Invalid or missing token"}
```

3.4 Get Job Detail

Method: GET | **URL:** `{{base_url}}/jobs/1` | **Role:** All roles

Returns complete details for a specific job. Replace '1' in the URL with the actual job ID. The response includes full user objects for created_by and assigned_to, timestamps, and the photo_path.

Headers:

```
Authorization: Bearer {{token}}
```

Success Response:

```
200 OK  
Returns a single full job object.
```

Error Responses:

```
404 Not Found: {"error": "Job not found"}
```

3.5 Assign Job

Method: PUT | **URL:** `{{base_url}}/jobs/1/assign` | **Role:** Manager only

Assigns or reassigns a job to a technician. Replace '1' in the URL with the job ID. The job status automatically changes to 'assigned'. Can be called multiple times for reassignment.

Headers:

```
Content-Type: application/json  
Authorization: Bearer {{token}}
```

Request Body:

```
{  
  "technician_id": 3  
}
```

Success Response:

```
200 OK  
Returns updated job object with status: 'assigned', the assigned_to object, and  
assigned_at timestamp.
```

Error Responses:

```
400 Bad Request: {"error": "technician_id required"}  
403 Forbidden: {"error": "Only managers can assign jobs"}  
404 Not Found: {"error": "Technician not found"}
```

3.6 Update Job Status

Method: PUT | **URL:** `{{base_url}}/jobs/1/status` | **Role:** Technician only

Updates job status. Technicians can only update their assigned jobs. The status flow is strictly: assigned → on_the_way → on_site → completed. Steps cannot be skipped or reversed.

Headers:

```
Content-Type: application/json
Authorization: Bearer {{token}}
```

Request Body:

```
{
  "status": "on_the_way"
}
```

Success Response:

```
{
  "id": 1,
  "status": "on_the_way"
}
```

Error Responses:

```
400 Bad Request: {"error": "Cannot go from assigned to completed",
  "next_allowed": "on_the_way"}
403 Forbidden: {"error": "This job is not assigned to you"}
403 Forbidden: {"error": "Only technicians can update status"}
```

3.7 Upload Job Photo

Method: POST | **URL:** `{{base_url}}/jobs/1/upload` | **Role:** CS or Manager

Uploads and attaches a photo to a job. Body must be multipart/form-data. In Postman: Select form-data, use exactly the key 'photo', change type to 'File', and select your image (jpg/png).

Headers:

```
Authorization: Bearer {{token}}  
(Do NOT manually set Content-Type; Postman handles the boundary boundary automatically)
```

Request Body:

```
Form Data:  
Key: photo  
Type: File  
Value: <Select File>
```

Success Response:

```
{  
  "photo_path": "job_1_photo.jpg"  
}
```

Error Responses:

```
400 Bad Request: {"error": "No file uploaded. Use key photo"}  
403 Forbidden: {"error": "Access denied"}
```

3.8 List Technicians

Method: GET | **URL:** `{{base_url}}/jobs/technicians` | **Role:** Manager only

Retrieves all users with the 'technician' role. Ideal for populating dropdown menus for job assignment.

Headers:

```
Authorization: Bearer {{token}}
```

Success Response:

```
[
  {
    "id": 3,
    "name": "Tech Ali",
    "email": "ali@demo.com",
    "role": "technician"
  },
  {
    "id": 4,
    "name": "Tech Reza",
    "email": "reza@demo.com",
    "role": "technician"
  }
]
```

Error Responses:

```
403 Forbidden: {"error": "Access denied"}
```

3.9 Dashboard Overview

Method: GET | **URL:** `{{base_url}}/dashboard` | **Role:** Manager only

Provides aggregate statistics for the manager dashboard, including counts by job status and workload distribution per technician.

Headers:

```
Authorization: Bearer {{token}}
```

Success Response:

```
{
  "total": 10,
  "pending": 3,
  "assigned": 2,
  "on_the_way": 1,
  "on_site": 1,
  "completed": 3,
  "by_technician": [
    {
      "technician": "Tech Ali",
      "job_count": 5
    },
    {
      "technician": "Tech Reza",
      "job_count": 3
    }
  ]
}
```

Error Responses:

```
403 Forbidden: {"error": "Access denied"}
```

3.10 Export Jobs CSV

Method: GET | **URL:** `{{base_url}}/dashboard/export/csv` | **Role:** Manager only

Generates and downloads a CSV export of all jobs. In Postman, click 'Save Response' -> 'Save to a file'.

Columns: ID, Title, Status, Location, Created By, Assigned To, Created At, Completed At

Headers:

```
Authorization: Bearer {{token}}
```

Success Response:

200 OK

Returns a file download with Content-Type: text/csv

Error Responses:

403 Forbidden: {"error": "Access denied"}

4. Recommended Testing Flow

Follow this sequence to thoroughly test the full system lifecycle:

#	Action	Expected Result / Note
1	POST /auth/login as cs@demo.com	Token auto-saves in environment
2	POST /jobs	Creates a job. Note the generated ID.
3	POST /jobs/1/upload	Attaches a photo to the created job.
4	POST /auth/login as manager@demo.com	Token updates to Manager role.
5	GET /jobs	Manager can view all jobs.
6	GET /jobs/technicians	Retrieves list of technician IDs.
7	PUT /jobs/1/assign	Assigns the job to technician_id 3.
8	GET /dashboard	View stats; the "assigned" count increases.
9	POST /auth/login as ali@demo.com	Token updates to Technician role.
10	GET /jobs	Technician sees ONLY their assigned job.
11	PUT /jobs/1/status	Body: {"status": "on_the_way"}
12	PUT /jobs/1/status	Body: {"status": "on_site"}
13	PUT /jobs/1/status	Body: {"status": "completed"}
14	POST /auth/login as manager@demo.com	Token updates to Manager role.
15	GET /dashboard/export/csv	Downloads CSV; job reflects as completed.

PART 2 — DATABASE DOCUMENTATION

Engine: SQLite

File Location: callmodule.db (located in the backend folder)

ORM: Flask-SQLAlchemy

Viewer Tool: DB Browser for SQLite (<https://sqlitebrowser.org>)

1. Table Schemas

Table: users

Column	Type & Modifiers	Description
id	INTEGER, PRIMARY KEY AUTOINCREMENT	Unique auto-generated identifier.
name	VARCHAR(100), NOT NULL	Full display name.
email	VARCHAR(120), UNIQUE NOT NULL	Login email (must be unique).
password	VARCHAR(200), NOT NULL	Bcrypt hashed password.
role	VARCHAR(20), NOT NULL	Roles: cs, manager, or technician.
created_at	DATETIME, DEFAULT now	Record creation timestamp.
updated_at	DATETIME, NULLABLE	Last update timestamp.

Seeded Demo Data: cs@demo.com (cs), manager@demo.com (manager), ali@demo.com (technician), reza@demo.com (technician). All default passwords are password123

Table: jobs

Column	Type & Modifiers	Description
id	INTEGER, PRIMARY KEY AUTOINCREMENT	Unique auto-generated identifier.
title	VARCHAR(200), NOT NULL	Short descriptive title.
description	TEXT, NOT NULL	Full task description.
location	VARCHAR(300), NULLABLE	Address or text location.
latitude	FLOAT, NULLABLE	GPS latitude (decimal degrees).

longitude	FLOAT, NULLABLE	GPS longitude (decimal degrees).
status	VARCHAR(30), DEFAULT pending	Current status. See flow below.
photo_path	VARCHAR(300), NULLABLE	Filename of uploaded photo (/uploads/).
created_by_id	INTEGER, NOT NULL (FK)	References users.id (CS who created).
assigned_to_id	INTEGER, NULLABLE (FK)	References users.id (Assigned tech).
created_at	DATETIME, DEFAULT now	Timestamp when created.
assigned_at	DATETIME, NULLABLE	Timestamp when assigned.
completed_at	DATETIME, NULLABLE	Timestamp when marked completed.
updated_at	DATETIME, NULLABLE	Timestamp of last record update.

2. Job Status Flow

Job statuses enforce a strict linear progression:

- **PENDING:** System default when a job is initially created by CS.
- **ASSIGNED:** Set by the Manager upon allocating a technician.
- **ON THE WAY:** Triggered by the Technician when driving to the site.
- **ON SITE:** Triggered by the Technician upon arriving at the location.
- **COMPLETED:** Triggered by the Technician when the task is resolved.

3. SQL Schema Definitions

```
CREATE TABLE users (  
    id            INTEGER      PRIMARY KEY AUTOINCREMENT,  
    name          VARCHAR(100) NOT NULL,  
    email         VARCHAR(120) UNIQUE NOT NULL,  
    password      VARCHAR(200) NOT NULL,  
    role          VARCHAR(20)  NOT NULL,  
    created_at    DATETIME,  
    updated_at    DATETIME  
);  
  
CREATE TABLE jobs (  
    id            INTEGER      PRIMARY KEY AUTOINCREMENT,  
    title         VARCHAR(200) NOT NULL,  
    description    TEXT        NOT NULL,  
    location      VARCHAR(300),  
    latitude      FLOAT,  
    longitude     FLOAT,  
    status        VARCHAR(30)  DEFAULT 'pending',  
    photo_path    VARCHAR(300),  
    created_by_id INTEGER      NOT NULL REFERENCES users(id),  
    assigned_to_id INTEGER      REFERENCES users(id),  
    created_at    DATETIME,  
    assigned_at   DATETIME,  
    completed_at  DATETIME,  
    updated_at    DATETIME  
);
```

4. How to View the Database

1. Download the Windows installer for DB Browser for SQLite from <https://sqlitebrowser.org/dl/>
2. Install and open the application.
3. Click Open Database, navigate to the backend folder, and select callmodule.db.
4. Use the Database Structure tab to review tables and columns.
5. Use the Browse Data tab and select a table from the dropdown to inspect records.
6. Use the Execute SQL tab to run custom SQL queries against the data.
7. To export the schema: Navigate to File → Export → Database to SQL file → save as schema.sql.